

Python Backend Clipping Migration Requirements

Summary

Move the Nest backend to Python/FastAPI. Keep the current analyze API and SSE progress flow working for the frontend.

Inside the backend, start using a clipping model. A YouTube video should become a long-form source with topic chunks, candidate clippings, coverage results, and run history.

Problem Frame

Reforge currently turns transcripts into categories and keywords. This works for the current UI. It is not strong enough for long videos.

The model can return good-looking categories while missing important parts near the end of a video. It can also miss side arguments, examples, and useful details.

The Python backend should improve LLM quality and make iteration faster. It should also avoid a frontend redesign in the first migration.

The backend should not decide too early what a clipping is for. A clipping may later become a startup idea, investment thesis, research note, learning note, or content outline. That decision belongs to a later user flow.

Key Decisions

- **Keep the outside compatible. Change the inside.** The API stays compatible. The backend stores richer analysis objects.
 - **Cover major topics, not every segment.** Every major topic shift should be handled. Every transcript segment does not need its own record.
 - **CandidateClipping is not SavedClipping.** Candidate clippings are generated by the system. Saved clippings will be user-owned later.
 - **Use one production model first.** Production uses one default provider/model. Internal tests can compare more models.
 - **Keywords are temporary compatibility.** The response still says keywords, but the backend should call these objects CandidateClipping internally.
-

Actors

- A1. **Current frontend user.** Uses the current analyze flow.
 - A2. **Future clipping user.** Will save, ignore, tag, organize, and develop clippings later.
 - A3. **Analysis pipeline.** Fetches, cleans, chunks, extracts, reviews, stores, and maps output.
 - A4. **Evaluator.** Compares model and prompt results on saved transcripts.
-

Requirements

External compatibility

- R1. The Python/FastAPI backend must keep the current analyze API behavior for the first migration.
- R2. The Python backend must keep SSE progress. It must stream useful stages and return the final compatible response.
- R3. The compatible response must map TopicChunk objects to today's categories.
- R4. The compatible response must map high-signal CandidateClipping objects to today's keywords.
- R5. Each returned category and keyword must trace back to stable internal TopicChunk and CandidateClipping IDs. The frontend may not use these IDs yet.

Durable internal model

- R6. The backend must store Source, Transcript, TopicChunk, CandidateClipping, and AnalysisRun records for each completed analysis.
- R7. Source must represent a long-form input, such as a YouTube video. It must not be only a request payload.
- R8. Transcript must keep cleaned text and source references. These are needed for citations, coverage review, and later reprocessing.
- R9. TopicChunk must represent one coherent major topic shift. It must keep timing or transcript references.
- R10. CandidateClipping must represent a system-generated candidate from an analysis run. It may include topic, claim, mechanism, risk, trend, entity, example, question, contradiction, signal level, and source reference.
- R11. AnalysisRun must record enough context to audit or compare results. It should include model settings, prompt version, schema version, transcript identity, validation status, and timing metadata.
- R12. The backend must keep richer CandidateClipping metadata internally, even if the current UI only uses part of it.

Long-form source pipeline

- R13. The pipeline must fetch the transcript, clean it, split it into major

topic chunks, extract candidate clippings, review coverage, store artifacts, and return the compatible response.

- R14. Chunking should create coherent topic chunks. It should not rely only on fixed-size text windows.
- R15. Extraction must keep candidates neutral. It must not force them into startup, investment, research, learning, or content categories.
- R16. Each candidate clipping must keep a timestamp or transcript reference.
- R17. Intros, sponsor reads, repeated content, and personal tangents should become low-signal or off-topic chunks when they are large enough to affect coverage.

Coverage discipline

- R18. A major topic shift is a meaningful change in subject, argument, examples, mechanism, named entity, risk, or practical implication. It must last long enough to form a coherent chunk.
- R19. Every major topic shift must have one of three outcomes: candidate clippings, a low-signal/off-topic chunk, or a coverage warning.
- R20. The pipeline must include an explicit coverage review after candidate extraction.
- R21. Coverage review must reduce missed late-video topics, side arguments, and important non-obvious sections.
- R22. Coverage review must not require segment-by-segment accounting in this migration.

Provider and evaluation behavior

- R23. Production analysis must use one configured default provider/model at first.
- R24. The first production version must not show users a model picker.
- R25. Internal evaluation must allow provider/model overrides.
- R26. The comparison harness must run the same transcript, chunks, prompt version, and schema version across model configs.
- R27. The harness should be ready to compare OpenAI, Claude, and Gemini, even if production uses only one model.
- R28. Evaluation metadata must include provider, model, prompt version, schema version, transcript hash, latency, estimated cost when available, validation errors, and raw output.

Key Flows

- F1. Compatible YouTube analysis
 - **Trigger:** The frontend sends an analyze request for a YouTube video.
 - **Actors:** A1, A3
 - **Steps:** The backend streams SSE progress. It fetches and cleans

- the transcript. It creates topic chunks, extracts candidate clippings, reviews coverage, stores artifacts, maps output to the compatible response, and emits the final result.
 - **Outcome:** The frontend gets the current categories-to-keywords response. The backend keeps richer clipping data.
 - **Covered by:** R1, R2, R3, R4, R5, R6, R13
 - F2. Coverage-aware extraction
 - **Trigger:** Chunking and first-pass clipping extraction are done.
 - **Actors:** A3
 - **Steps:** Coverage review compares topic chunks with extracted candidates. It marks low-signal/off-topic chunks. It records warnings for chunks it cannot classify with confidence.
 - **Outcome:** Major topic shifts are accounted for without tracking every segment.
 - **Covered by:** R18, R19, R20, R21, R22
 - F3. Internal model comparison
 - **Trigger:** An evaluator runs the harness on a saved transcript or fixture.
 - **Actors:** A4
 - **Steps:** The harness runs selected model configs on the same transcript, chunks, prompt version, and schema version. It stores metadata and raw outputs.
 - **Outcome:** The team can compare prompt and model quality without changing production.
 - **Covered by:** R23, R25, R26, R27, R28
-

Acceptance Examples

- AE1. **Covers R3, R4, R5.** Given an analysis with three topic chunks and eight high-signal candidate clippings, when the response is returned, then categories map to topic chunks, keywords map to candidate clippings, and each item traces to an internal ID.
- AE2. **Covers R18, R19, R20.** Given a transcript that shifts to a founder example near the end, when extraction finishes, then that section appears as a candidate clipping, a low-signal/off-topic chunk, or a coverage warning.
- AE3. **Covers R15, R16.** Given a high-signal chunk about a company using a distribution mechanism, when extraction runs, then the candidate is stored neutrally with source references. It is not forced into a startup, investment, research, or content category.
- AE4. **Covers R23, R24, R25.** Given production uses one default model, when a normal analyze request runs, then the user does not choose a model. When an internal eval runs, provider/model override is available.
- AE5. **Covers R28.** Given one provider fails output validation, when metadata is saved, then the run records provider, model, prompt version,

schema version, transcript hash, latency, available cost estimate, validation errors, and raw output.

Success Criteria

- The current frontend can use the Python backend without a major redesign.
 - Long YouTube videos are less likely to miss major topic shifts.
 - Candidate clippings are durable enough for later save, ignore, tag, organize, and develop flows.
 - Prompt and model changes can be compared on saved transcripts.
 - The backend uses CandidateClipping internally instead of building the new model around keyword terminology.
-

Scope Boundaries

Deferred for later:

- User save, ignore, tag, and organization decisions.
- SavedClipping and user-owned decision state.
- Collections and boards.
- Developed startup, investment, research, learning, or content artifacts.
- Full provider parity in production.
- Full job polling.
- Major frontend redesign.

Outside this migration:

- Segment-by-segment transcript accounting.
 - A user-facing model picker.
 - Treating generated candidate clippings as user-saved clippings.
-

Dependencies / Assumptions

- The Python backend can keep the current analyze contract close enough for the existing frontend.
 - SSE progress is enough for the first coverage-aware pipeline.
 - One default production model can meet the first quality bar.
 - The comparison harness can guide future model choices.
 - Stable internal IDs can be created before the frontend depends on them.
-

Sources / Research

- `backend-nest/src/analyze/analyze.service.ts` coordinates request parsing, transcript resolution, transcript caching, LLM analysis, and response construction.
- `backend-nest/src/analyze/youtube.service.ts` already delegates YouTube transcript fetching to Python.
- `backend-nest/src/llm/llm.service.ts` handles prompt execution, JSON parsing, source validation, and timestamp URL resolution.
- `backend-nest/src/transcript/transcript-store.service.ts` stores transcripts in memory with TTL. This is not enough for durable analysis objects.
- `backend-nest/README.md` documents the current public API: GET `/health`, POST `/analyze`, SSE progress for `analyze`, and GET `/transcript/:transcriptId`.